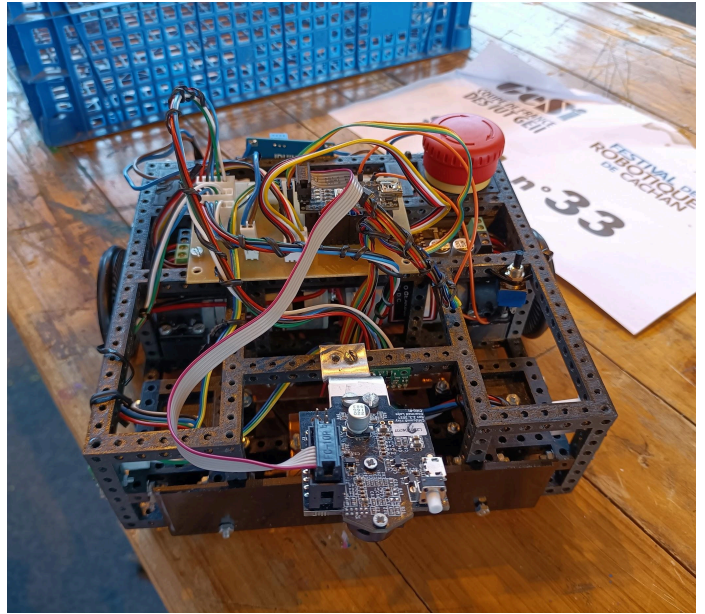
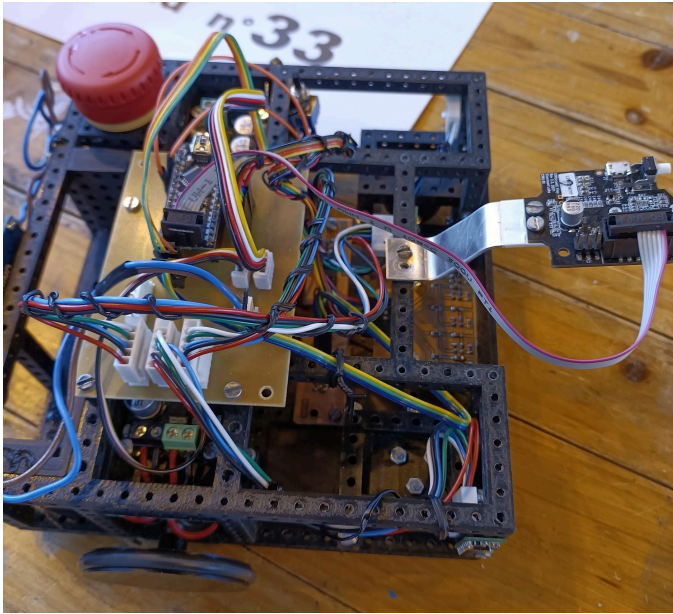


Compte Rendu SAE Robot Suiveur de Ligne

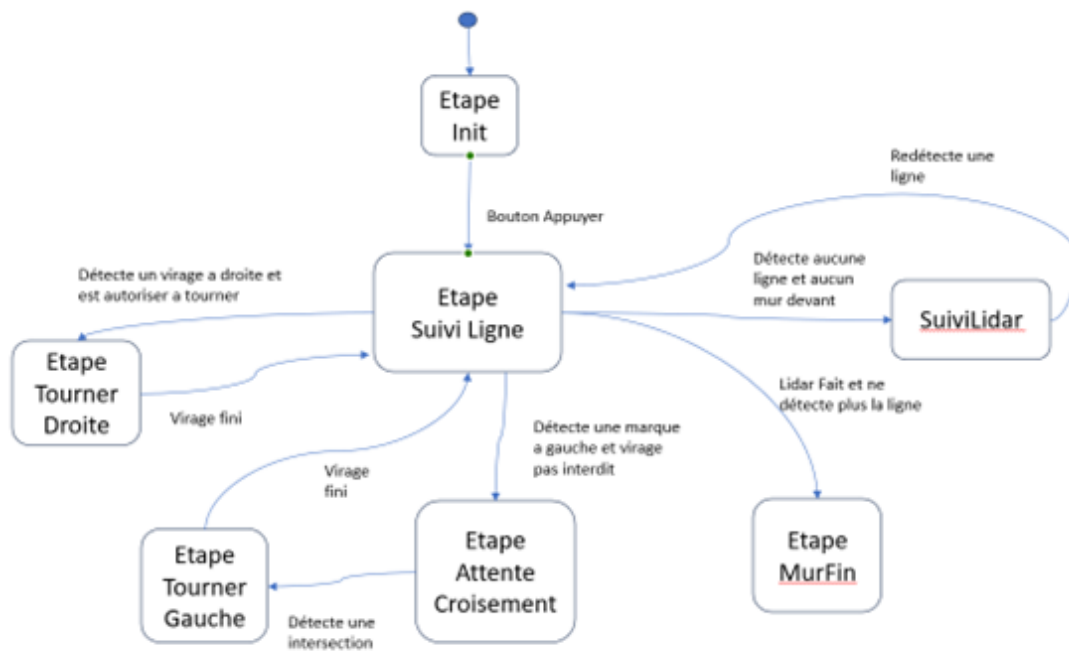


Objectif

Le robot suiveur de ligne : l'objectif de ce projet était de réaliser un robot suiveur de ligne. On nous a mis à disposition le châssis, les moteurs et les contrôleurs Cytron pour gérer les moteurs. À partir de ces éléments, nous avons dû, en groupe de trois personnes, réaliser ce projet. Nous avons dû apprendre à travailler en groupe car, en effet, nous nous sommes divisés les tâches par trois. Adrien Michel s'est occupé de la partie des capteurs Lidar, Amine Atamna s'est occupé d'utiliser la caméra Pixy et Antoine Passier s'est occupé des capteurs CNY70. Nous avons travaillé chacun sur notre partie tout en nous entraïdant pour avancer le plus loin possible dans ce projet.

Première partie les capteurs Lidar par Adrien Michel

Diagramme d'état :



Comment suivre des murs avec les capteurs Lidar ?

Les capteurs Lidars sont des capteurs de distances, ceux-ci nous donnent donc une idée de où le robot se trouve. Nous pouvons donc utiliser le capteur pour que le robot reste à une distance donnée du mur. Si il est trop proche celui-ci doit s'éloigner et vice versa

- Pour faire ce système, nous avons soustrait la valeur consigne (150 mm) à la valeur du capteur. De cette façon, lorsque le capteur donne une valeur négative, nous savons que le robot est trop proche du mur.
- Cette valeur, nommée V_r , est ensuite ajoutée à une autre valeur, nommée V_t , qui elle est constante. La valeur V_t , fixée à 400, donne une vitesse constante au robot, tandis que la valeur V_r fait tourner le robot

- L'inconvénient de cette technique est que le robot tourne très peu lorsqu'il est trop proche du mur, et au contraire, peut tourner extrêmement rapidement lorsqu'il est trop loin de celui-ci. Pour remédier à ce problème, une valeur maximale de V_r égale à $V_t / 2$, ici 200, a été instauré
- Pour contrecarrer cela un coefficient a été rajouté: la vitesse de rotation V_r est constamment divisée de moitié et si la valeur de V_r est négative, donc que le robot est trop proche du mur, la valeur est multipliée par 4. Cela revient à doubler la valeur, sans la diviser.

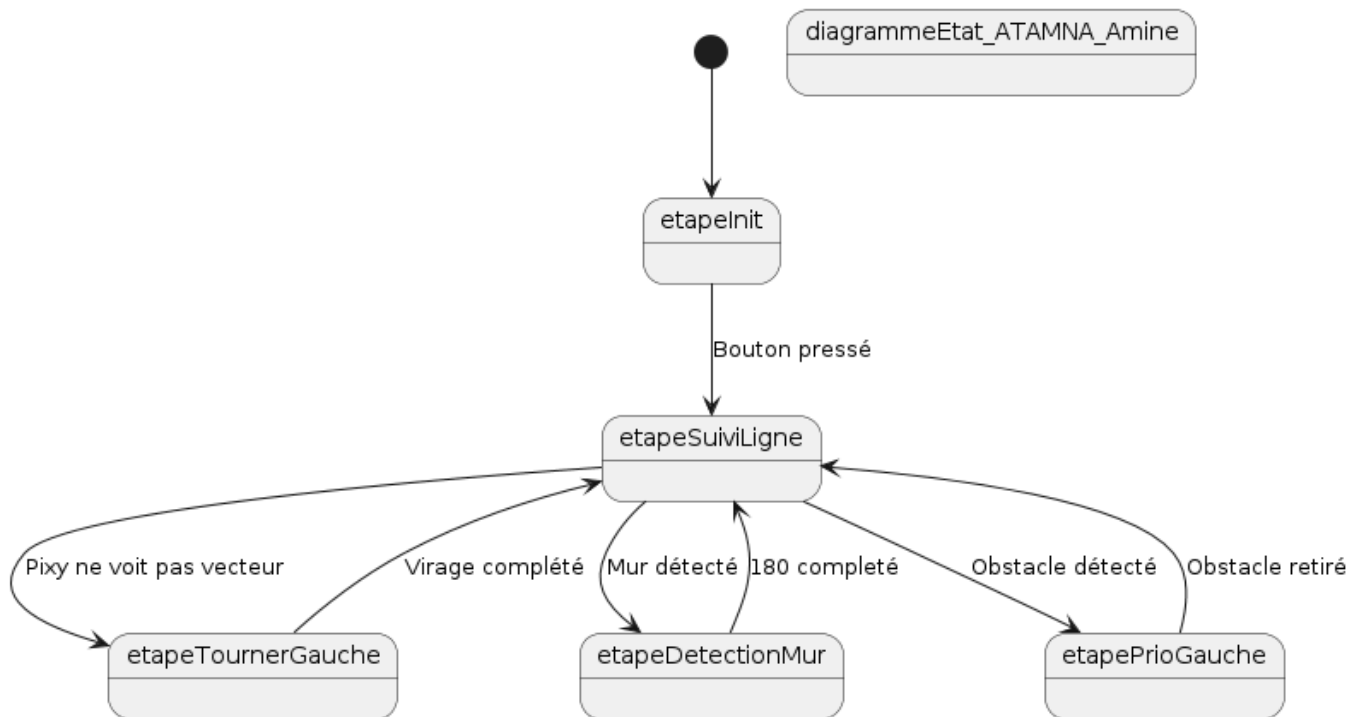
Ce système fonctionne, et permet au robot de suivre un mur à la fois. Pour permettre au robot de suivre plusieurs murs nous devons pouvoir changer quel capteur fait la mesure.

- Pour cela, nous comparons les valeurs des capteurs, tout en donnant la priorité au mur de droite. Si le capteur de droite donne une distance par rapport au robot supérieur à 600 mm, cette mesure est ignorée. Si le capteur de gauche donne une valeur inférieure, alors c'est cette mesure qui est utilisée, et le robot suit le mur de gauche.
- Cependant il peut arriver que les capteurs donnent tous deux des valeurs supérieures à 600. Dans ce cas de figure, le robot est ordonné de continuer tout droit.

Les valeurs faisant tourner les moteurs sont utilisées dans la fonction `vitesseMoteur`. Cette fonction a pour but de vérifier si les valeurs donné n'excède pas les vitesse maximale des moteurs (1600) mais aussi de donner la vitesse de chaque moteur: pour le moteur Gauche, le calcul est $(V_t + V_r)$ tandis que pour le moteur Droit le calcul est $(V_t - V_r)$.

Seconde partie la caméra Pixy par Amine Atamna

Diagramme d'état :



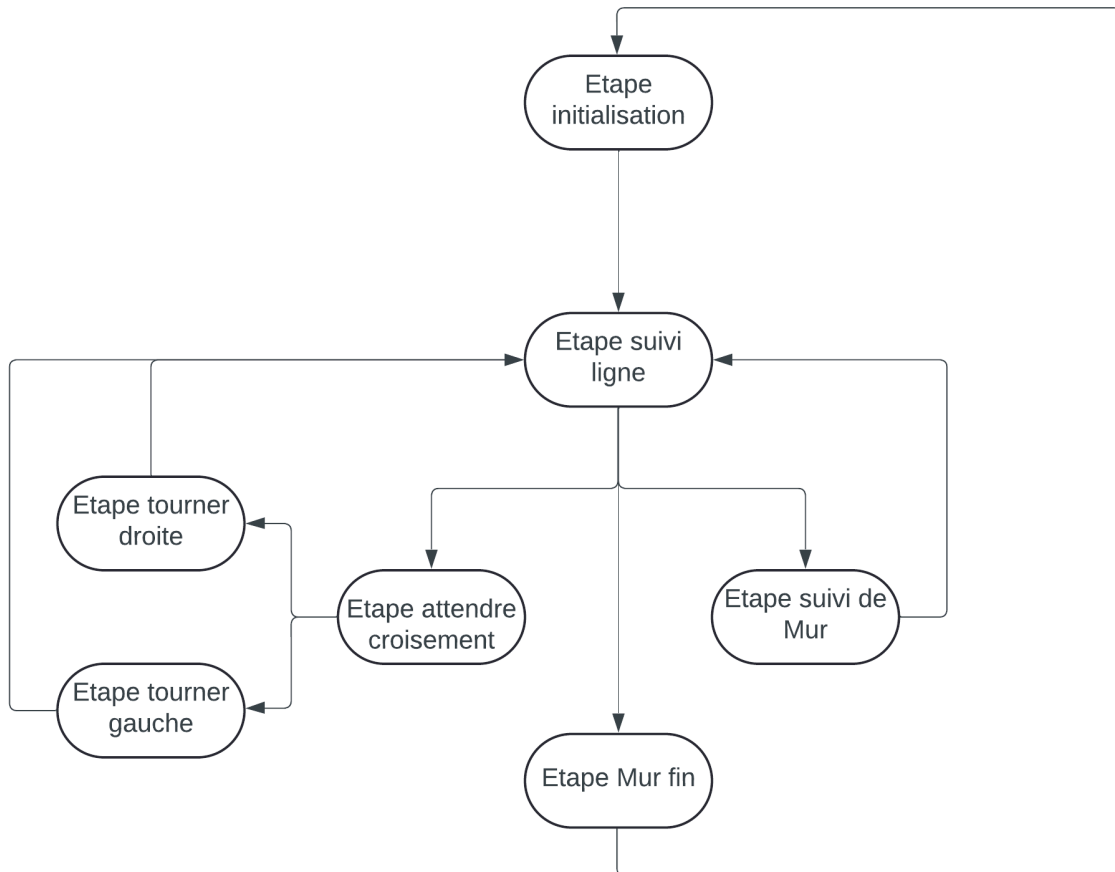
Comment suivre la ligne avec la caméra pixy ?

- La stratégie de suivi de ligne avec la caméra Pixy repose sur l'utilisation de la détection de lignes et de vecteurs pour ajuster en temps réel la trajectoire du robot. La Pixy identifie chaque ligne sous forme de vecteurs. Un vecteur représente une ligne détectée avec des coordonnées de début et de fin.
- Pour recentrer les coordonnées et corriger un éventuel décalage lié à la position de la caméra sur le robot, les coordonnées x du début et de la fin du vecteur sont ajustées en soustrayant une valeur fixe (ici, 39 car les abscisses vont de 0 à 79) de chaque coordonnée x.
- La différence entre les coordonnées x du début et de la fin du vecteur. Cette différence indique l'inclinaison de la ligne détectée. En utilisant cette différence et la position x du début du vecteur, nous calculons la valeur de correction v_r , cette valeur représente la déviation du robot par rapport à la ligne centrale et sera utilisée pour ajuster la trajectoire du robot.

- La vitesse de translation v_t est définie en fonction de la valeur de v_r . Si v_r est inférieur ou égal à 10 (la ligne est horizontale) , v_t est fixée à 400 ; sinon, elle est fixée à 300. Cela permet d'ajuster la vitesse du robot en fonction de la déviation détectée avec une vitesse adaptée aux virages . On utilise ensuite une correction proportionnelle pour ajuster les vitesses des moteurs gauche et droit. Les vitesses des moteurs sont définies comme suit :
 $vitesse_moteur_gauche = v_t + k_p * v_r$ et $vitesse_moteur_droit = v_t - k_p * v_r$, où k_p est le gain proportionnel qui détermine l'importance de la correction appliquée.
- La fonction `vitesseMoteur` applique les vitesses calculées aux moteurs du robot. Cette étape ajuste dynamiquement la direction et la vitesse du robot, lui permettant de suivre la ligne détectée par la Pixy. Grâce à cette stratégie, le robot peut suivre une ligne de manière autonome, en réagissant précisément et rapidement aux courbes et changements de direction de la ligne. Les ajustements continus assurent que le robot reste sur la trajectoire désirée malgré les perturbations.

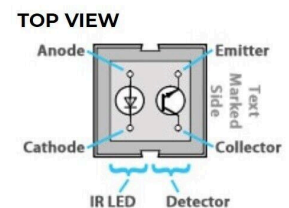
Troisième partie les capteurs CNY70 par Antoine Passier

Diagramme d'état :

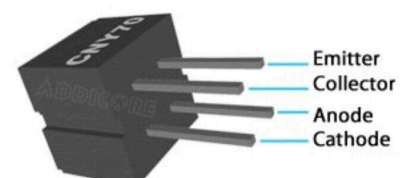


Suivi de Ligne avec les capteurs CNY70 :

Le principe de fonctionnement des capteurs CNY70 repose sur la réflexion de la lumière infrarouge émise par une LED sur une surface. Plus la surface est claire, plus la lumière est réfléchiée, et le phototransistor laisse passer un courant plus important. C'est de cette façon que l'on différencie une surface blanche d'une ligne noire.



PINOUT



Afin de corriger la trajectoire de façon fluide, j'ai choisi d'utiliser 5 capteurs CNY70 pour obtenir différents facteurs de correction. Pour permettre une lecture la plus rapide possible avec les capteurs, j'ai utilisé une méthode de lecture digitale : niveau logique 1 si le capteur est sur du noir et niveau logique 0 si le capteur est sur du blanc. Pour cela, j'ai utilisé un condensateur et une résistance. Ainsi, avec une simple lecture digitale des 5 pins correspondant aux 5 capteurs, on peut effectuer une prise de mesure en un minimum de temps et mesurer les 5 capteurs simultanément.

Le principe est simple : lorsque je mets le pin de mon Arduino en sortie à l'état 1, le condensateur se décharge. Une fois déchargé, je repasse le pin en entrée pour que le condensateur se charge. C'est en fonction du temps que mettra le condensateur à se charger que l'on peut déterminer si l'on est au-dessus de la ligne noire, car le phototransistor fera passer plus ou moins de courant selon qu'il est au-dessus du noir ou du blanc.

Il est possible, en fonction de la réflexion du sol blanc et noir, d'adapter les capteurs pour les calibrer en modifiant les temps de charge et de décharge des condensateurs.

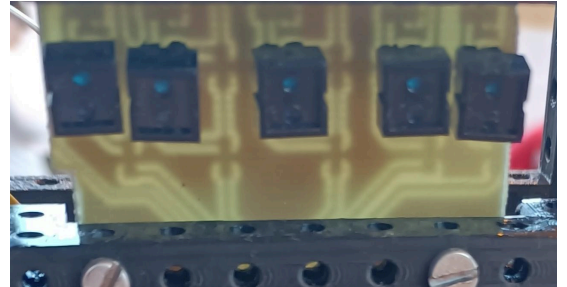
Voici la fonction correspondante :

```
void mesurePhotoElectrique() {  
    // Je met les 5 pin des capteurs en sortie  
    DDRD |= (1 << PD2) | (1 << PD3) | (1 << PD4) | (1 <<  
    PD5) | (1 << PD6);  
    // Je met les 5 sortie au niveau haut  
    PORTD |= (1 << PD2) | (1 << PD3) | (1 << PD4) | (1 <<  
    PD5) | (1 << PD6);  
    // Temps de charges des condensateur  
    _delay_us(tempsDechargeCondensateur);  
    // Je met les 5 pin des capteurs en entree  
    DDRD &= ~(1 << PD2) | (1 << PD3) | (1 << PD4) | (1 <<  
    PD5) | (1 << PD6));  
    // Je met les 5 sortie au niveau bas  
    PORTD &= ~(1 << PD2) | (1 << PD3) | (1 << PD4) | (1 <<  
    PD5) | (1 << PD6));  
    // Temps de décharges des condensateur  
    _delay_us(tempsChargeCondensateur);  
}
```

```
// Je lis les valeurs du port D
portDValue = PIND & 0b01111100;
}
```

La correction de la trajectoire repose sur le positionnement des capteurs: plus un capteur est placé à l'extrémité, plus la correction sera élevée. Il existe 9 cas possibles le cas où seul le capteur du milieu est au-dessus de la ligne noire (le cas normal où le robot est aligné avec la ligne) et les autres cas dépendent de la position des capteurs gauche, centre et droite par rapport à la ligne noire. Par

exemple, si le capteur gauche et le capteur central sont sur la ligne, ou si seulement le capteur gauche, ou encore si le capteur gauche et l'extrême gauche, ou uniquement l'extrême gauche, et bien sûr de même pour les capteurs de droite.



De plus, l'équation de la trajectoire est calculée à partir d'un pourcentage en fonction de la vitesse nominale et de l'un des neuf cas possibles (valeur correspondant à la position du capteur), qui est ensuite multiplié par un coefficient multiplicateur k. Voici l'équation de correction :

$$wr = ((VT * position) / 100) * k$$

Voici la fonction de mon programme qui gère ces corrections :

```
void calculTrajectoire() {
    switch (portDValue) {
        case 0b00000100: // gauchePPP
            position = 20;
            break;
        case 0b00001100: // gauchePP
            position = 16;
            break;
        case 0b00001000: // gaucheP
            position = 8;
            break;
        case 0b00011000: // gauche
            position = 4;
            break;
        case 0b00010000: // Centre
```



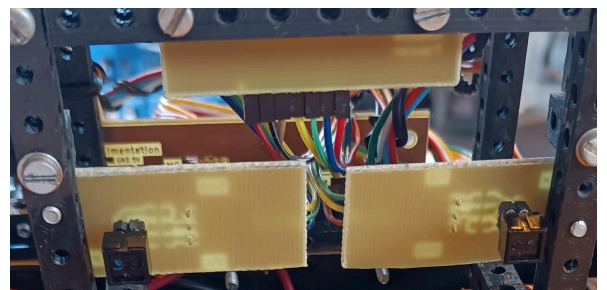
```

    position = 0;
    break;
case 0b00110000: // droit
    position = -4;
    break;
case 0b00100000: // droitP
    position = -8;
    break;
case 0b01100000: // droitPP
    position = -16;
    break;
case 0b01000000: // droitPPP
    position = -20;
    break;
}
// Equation correction trajectoire
wr = ((VT * position) / 100) * k;
}

```

Détection de marque avec les capteurs CNY70 :

L'une des parties les plus difficiles fut de prendre des virages à gauche ou à droite. En effet, il a fallu utiliser deux capteurs photoréfléctifs supplémentaires pour détecter les marques gauche et droite. Une fois celles-ci détectées, il fallait tourner à la prochaine intersection. La fonction de virage à droite ou à gauche fonctionne plutôt bien, mais de nombreux problèmes ont été rencontrés quant à la détection des marques gauche ou droite. Si l'un des capteurs détectent un croisement à gauche ou à droite, le robot entrait en mode d'attente de croisement et attendait donc la prochaine intersection pour tourner.



J'ai donc contourné ce souci avec un `triggerTime` qui permet, après la détection d'une marque, de réinitialiser la variable de virage à gauche ou à droite si aucun croisement n'a été détecté après 800 millisecondes. Ainsi, le robot sort du mode d'attente de croisement. Cette fonction a également résolu le problème d'un passage sur un croisement tout droit, où les deux capteurs détectent une marque pendant un très court instant, ce qui fait parfois tourner le robot dans le mauvais sens ou le fait tourner sans nécessité.

Voici le code correspondant :

```
case etapeAttenteCroisement:
    VT = valeurVitesse;
    calculTrajectoire(); // Fonction calcul de la trajectoire
    vitesseMoteur((VT + wr), (VT - wr));
    static bool passageBlancGauche = false, passageBlancDroit
= false;
    if (bit_is_clear(PINC, PC3)) passageBlancGauche = true;
    if (bit_is_clear(PINC, PC2)) passageBlancDroit = true;
    if (triggerTime1 + 800 >= currentTime -
tempsEcoulerPrioriteGauche) // Je teste si je vois une
intersection avant les 800 millisecondes alors j'autorise de
faire le virage
        // currentTime s'incrémente en permanence de 1 toutes les
millisecondes, tempsEcoulerPrioriteGauche est une variable qui
donne le temps écouler pendant que le robot est bloquer (avec la
priorite gauche)
        {
            if ((bit_is_set(PINC, PC3)) && (bit_is_clear(PINC, PC2))
&& (passageBlancGauche) && (tourne == true)) {
                passageBlancGauche = false;
                etapeSuivante = etapeTournerGauche; // je lance
l'etape tourner a gauche
                triggerTime1 = 0; // je
réinitialise la variable temps qui me dis si je peux ou pas faire
un virage
            } else if ((bit_is_set(PINC, PC2)) &&
(bit_is_clear(PINC, PC3)) && (passageBlancDroit) && (tourne ==
false)) {
                passageBlancDroit = false;
                etapeSuivante = etapeTournerDroite; // je lance
l'etape tourner a droite
                triggerTime1 = 0; // je
réinitialise la variable temps qui me dis si je peux ou pas faire
un virage
            }
        } else etapeSuivante = etapeSuiviLigne; // je reprend le
suivi de ligne
        break;
```

Suivi de Mur avec le Lidar :

Enfin, l'association de mon programme avec le capteur lidar, un capteur simple à utiliser qui renvoie la distance à laquelle je me trouve du mur, n'a pas été si difficile. J'ai ajouté à mon code que lorsque je détecte uniquement du blanc, je passe en mode Suivi de Mur et utilise alors le lidar pour me déplacer jusqu'à retrouver un bout de ligne, moment où je récupère le suivi de ligne classique. Pour mon suivi de mur, je corrige ma trajectoire de façon proportionnelle : plus je m'éloigne, plus je corrige, et inversement, plus je me rapproche, plus je corrige, ce qui permet un suivi de mur assez fluide et rapide.

Voici le code correspondant :

```
void correctionLidar(uint16_t distance, uint16_t consigne) {  
    VT = 350;  
    if (distance >= consigne) wr = map(distance, consigne, consigne  
+ 40, 10, 60);  
    else wr = map(distance, consigne - 40, consigne, -60, -10);  
}
```

Nous avons également dû implémenter une fonction permettant d'utiliser les fonctionnalités Arduino, sans laquelle il aurait été impossible d'utiliser le Lidar.

BILAN de ce projet robot suiveur de ligne :

Lors de ce projet de robot suiveur de ligne , plusieurs défis techniques ont été rencontrés. Grâce à des solutions spécifiques et des améliorations, nous avons pu optimiser le système pour un fonctionnement plus précis et fiable.

Un des premiers problèmes rencontrés concerne la détermination du bon coefficient proportionnel (k_p). Ce coefficient est crucial pour le contrôle des moteurs, car un k_p trop élevé peut entraîner des oscillations excessives, tandis qu'un k_p trop faible ne fournirait pas une correction suffisante pour maintenir le robot sur la ligne. Pour résoudre ce problème, des tests ont été réalisés en ajustant progressivement le k_p et en observant le comportement du robot. Cette méthode nous a permis d'identifier la valeur optimale qui stabilise le robot sans provoquer d'oscillations excessives.

Un autre défi important était de trouver la meilleure position et inclinaison pour la caméra Pixy. La position de la caméra affecte directement la précision de la détection des vecteurs de ligne. Après plusieurs essais, nous avons découvert que placer la caméra presque parallèle au sol améliore considérablement la précision de la détection.

En voulant combiner les trois capteurs (LIDAR,CNY,PIXY) , nous avons constaté des différences dans les fonctions des moteurs et dans la structure de nos programmes . Pour remédier à cela, il aurait été préférable de créer une partie commune du programme dès le début du projet. Cela aurait facilité le partage de données dans notre trinôme.

Nous avons également rencontré des erreurs lors du routage initial de la carte Shield Nano, où les ports de la Nano avaient été inversés.. Pour corriger ces erreurs, un deuxième routage de la carte Shield a été réalisé, alignant correctement les ports et rétablissant une fonctionnalité adéquate. Cette correction a été essentielle pour assurer le bon fonctionnement de l'ensemble du système.

Un autre problème était de trouver le meilleur emplacement pour les capteurs CNY supplémentaires. Ceux-ci devaient pouvoir détecter les marques sur les côtés, sans détecter la ligne, tout en étant le plus proche possible des autres capteurs.

L'utilisation de ces mêmes capteurs posait eux aussi problème. Étant donné que ceux-ci ne détectent pas les doubles marques en même temps, ils considéraient ces doubles marques comme une indication de virage, à gauche ou à droite, selon quel capteur atteignait la marque en premier.

Pour améliorer davantage le système, nous pouvons envisager l'intégration d'un étalonnage automatique des capteurs. Cette fonctionnalité permettrait de compenser les variations de luminosité, garantissant une détection précise des lignes même dans des conditions d'éclairage changeantes. Un système d'étalonnage automatique ajuste les seuils de détection en temps réel, assurant ainsi une performance optimale du robot indépendamment des variations de l'environnement.

En conclusion, les défis ont été surmontés grâce à des ajustements techniques précis et des tests rigoureux. En déterminant le bon coefficient proportionnel, en ajustant la position de la caméra, en standardisant les fonctions et en corrigeant les erreurs de routage, nous avons considérablement amélioré la performance du système. L'ajout potentiel d'un étalonnage automatique des capteurs pourrait encore renforcer la robustesse du robot face aux variations de luminosité. Ces solutions et améliorations sont essentielles pour garantir que le robot puisse suivre les lignes de manière autonome et précise.

Chiffrage du matériel et des ressources humaines :

Matériel nécessaire pour le robot

- 2 moteurs + 2 roues
- carcasse globale du robot
- carte arduino nano
- Une caméra Pixy
- 3 capteurs Lidar (1 pointé à gauche, 1 pointé à droite, 1 pointer devant)
- 7 capteurs photoréfléctif CNY (5 devant pour suivre la ligne, et 2 autre plus en retrait pour repérer les marque au sol
- fils nécessaire au branchement

Ressource Humaine utilisée pour le développement : 3 personnes pour une durée de 3 semaines